Introductory Machine Learning: Assignment 3

Deadline:

Assignment 3 is due Thursday, March 26 at 11:59pm. Late work will not be accepted as per the course policies (see the Syllabus and Course policies on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The problems are broken up into steps that should help you to make steady progress.

Submission:

Submit your assignment as a .pdf on Gradescope, and as a .ipynb on Canvas. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered.

Note:

1. Please **assign each sub-question on Gradescope** correctly. This ensures all questions are visible and helps TAs grade efficiently.
2. Double-check that **your PDF is complete** before submission.
3. For both text and mathematical answers, please use **markdown blocks** instead of code block notes. This prevents answers from being cut off and improves readability. Also, ensure you run every cell before knitting, as un-run markdown cells may have formatting issues.

To produce the .pdf, please convert to html and then print to pdf. (You may want to use your pdf print menu to scale the pages to be sure that cells are not truncated.) To convert to html, you can use this converter notebook.

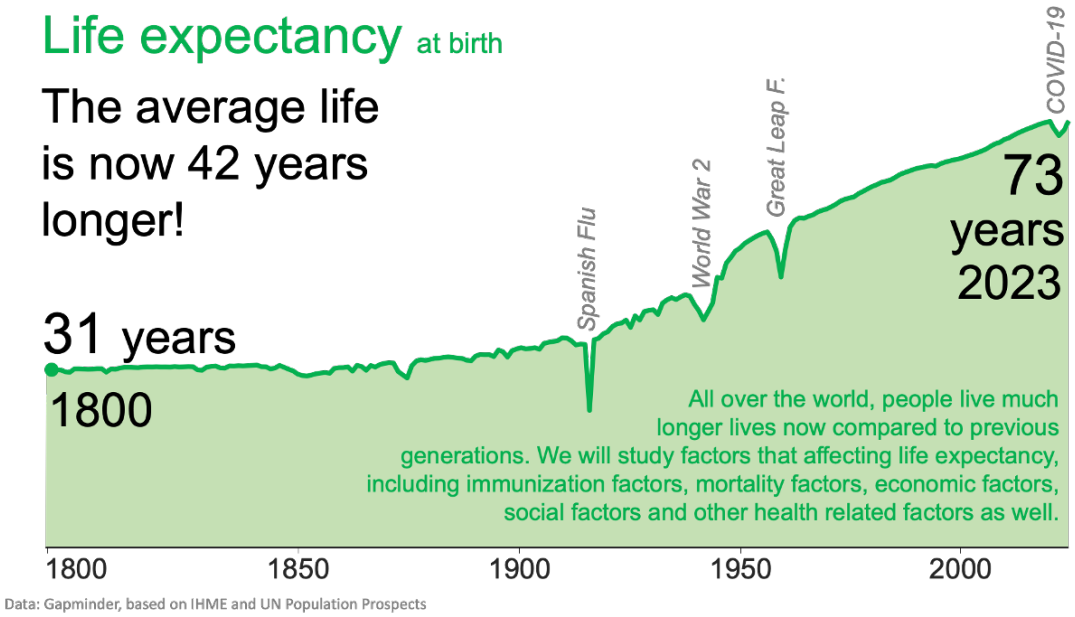
Topics

1. Random forests
2. Principal components analysis
3. Bayes' theorem

In [78]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
%matplotlib inline
```

Problem 1: Identifying factors impacting global life expectancy through the trees (25 points)

The Global Health Observatory (GHO) under the World Health Organization (WHO) tracks health status and related factors for all countries. Their life expectancy dataset includes variables like immunization, mortality, and economic and social factors, helping to identify key areas for improving a country's life expectancy.



In this problem, we'll leverage the power of regression decision trees and regression forests random forests to explore the factors influencing life expectancy and predict it. Are you ready to dive into the data and see how we might live longer?

In [79]:
```
# Load the dataset. Don't change it.
life_expectancy = pd.read_csv("https://raw.githubusercontent.com/YData123
life_expectancy = life_expectancy.drop(columns = [' thinness  1-19 years'
life_expectancy.rename(columns={'Life expectancy ': 'Life expectancy', 'M
                               'Diphtheria ': 'Diphtheria', ' HIV/AIDS':
life_expectancy = life_expectancy.dropna()
life_expectancy.head()
```

Out[79]:

	Country	Year	Status	Life expectancy	Adult Mortality	Infant deaths	Alcohol	Percentage expenditure
0	Afghanistan	2015	Developing	65.0	263.0	62	0.01	71.279624
1	Afghanistan	2014	Developing	59.9	271.0	64	0.01	73.523582
2	Afghanistan	2013	Developing	59.9	268.0	66	0.01	73.219243
3	Afghanistan	2012	Developing	59.5	272.0	69	0.01	78.184215
4	Afghanistan	2011	Developing	59.2	275.0	71	0.01	7.097109

You can find the desciption of each column as follows:

- `Country` : Country
- `Year` : Year

- `Status` : Country Developed or Developing status
- `Life expectancy` : Life expectancy in age
- `Adult Mortality` : Adult Mortality Rates of both sexes (probability of dying between 15 and 60 years per 1000 population)
- `Infant deaths` : Number of Infant Deaths per 1000 population
- `Alcohol` : Alcohol, recorded per capita (15+) consumption (in litres of pure alcohol) -percentage expenditure: Expenditure on health as a percentage of Gross Domestic Product per capita(%)
- `Percentage expenditure` : Expenditure on health as a percentage of Gross Domestic Product per capita(%)
- `Hepatitis B` : Hepatitis B (HepB) immunization coverage among 1-year-olds (%)
- `Measles` : Measles - number of reported cases per 1000 population
- `BMI` : Average Body Mass Index of entire population
- `Polio` : Polio (Pol3) immunization coverage among 1-year-olds (%)
- `Diphtheria` : Diphtheria tetanus toxoid and pertussis (DTP3) immunization coverage among 1-year-olds (%)
- `HIV/AIDS` : Deaths per 1000 live births HIV/AIDS (0-4 years)
- `GDP` : Gross Domestic Product per capita (in USD)
- `Population` : Population of the country
- `Income composition of resources` : Human Development Index in terms of income composition of resources (index ranging from 0 to 1)
- `Schooling` : Number of years of Schooling(years)

We will focus on the numerical data. We first select the predictor variables X and the response variable 'Life expectancy'. Then we use `train_test_split` to randomly split the data into training and test sets.

In [80]:
```
# Just run the cell.
life_expectancy = life_expectancy.drop(columns = ['Country', 'Status'])

from sklearn.model_selection import train_test_split
y = life_expectancy['Life expectancy']
X = life_expectancy.drop('Life expectancy', axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4,
```

1.1 Building a simple regression tree

Let's start with two of the predictor variables: `HIV/AIDS` (Deaths per 1 000 live births HIV/AIDS (0-4 years)) and `Income composition of resources` (Human Development Index in terms of income composition of resources). You will *manually* build a regression tree using these two variables step by step.

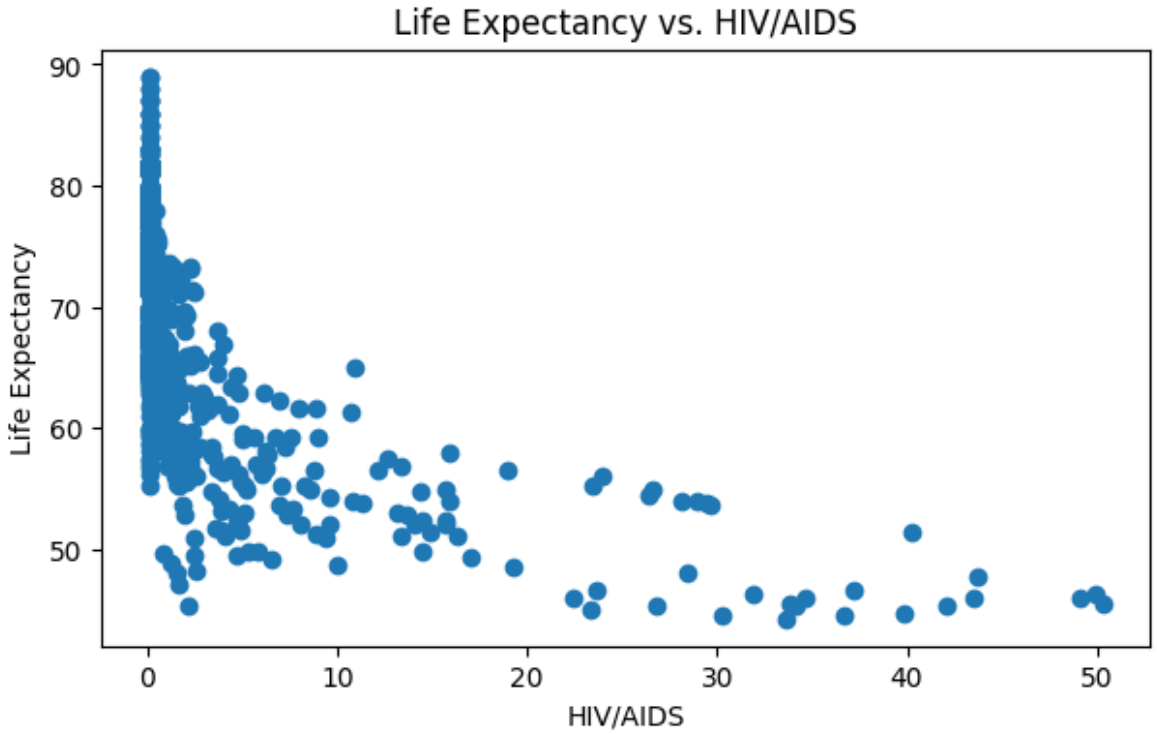
Problem 1.1.a

First, make two scatter plots: 'Life Expectancy' vs. 'HIV/AIDS' and 'Life Expectancy' vs. 'Income Composition of Resources'.

In [81]:
```
hiv = X_train['HIV/AIDS'].values
```

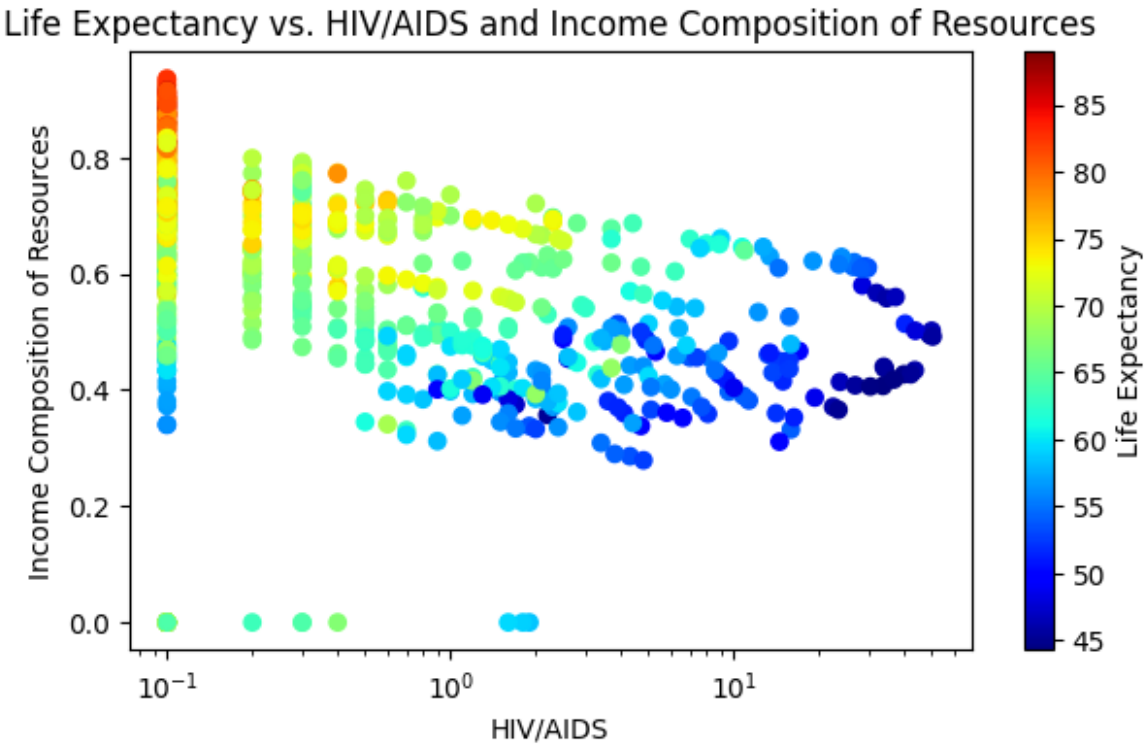
```
income = X_train['Income composition of resources'].values
expectancy = y_train.values

### your code starts here

plt.figure(figsize=(7, 4))
scatter = plt.scatter(hiv, expectancy)
plt.title('Life Expectancy vs. HIV/AIDS')
plt.xlabel('HIV/AIDS')
plt.ylabel('Life Expectancy')
plt.figure(figsize=(7, 4))
scatter = plt.scatter(income, expectancy)
plt.title('Life Expectancy vs. Income Composition of Resources')
plt.xlabel('Income Composition of Resources')
plt.ylabel('Life Expectancy')
```

Out[81]: Text(0, 0.5, 'Life Expectancy')



We can plot Life Expectancy indicated by color using a colormap, and see how life expectancy depends on these two.

In [68]:
```
# Just run it, no need to change.
plt.figure(figsize=(7, 4))
scatter = plt.scatter(hiv, income, c=expectancy, cmap='jet')
plt.title('Life Expectancy vs. HIV/AIDS and Income Composition of Resourc
plt.xlabel('HIV/AIDS')
plt.ylabel('Income Composition of Resources')
cbar = plt.colorbar(scatter)
cbar.set_label('Life Expectancy')
plt.xscale("log")
plt.show()
```



What can you observe from these plots? How do you think this could help in building decision trees? Briefly explain your thoughts in 2-3 sentences.

I can see from the first two scatter plots that life expectancy decays exponentially with increase in HIV/AIDS deaths and has a positive linear relationship with income comp. From the colormap plot, it looks like there is pretty clear gradient of decrease in life expectancy as income dec and HIV/AIDS deaths increase, and income and hiv/aids deaths also has a negative exponential curve. The non-linearity of data and gradient of life expectancy will help in building the decision tree since it splits the data into segments.

Problem 1.1.b

Next, we can define a function to find the best split for a simple regression tree based on a single feature to minimize the mean squared error (MSE). This function is provided for you, and it can be helpful for a fundamental understanding of building a decision tree from scratch.

- feature (np.array): The feature data used for finding the split (e.g. `hiv`).
- target (np.array): The target data used for calculating MSE (e.g. `expectancy`).

Use the `find_best_split` function provided to find the best split and the MSE using 'HIV/AIDS' and 'Income Composition of Resources'.

In [69]:
```
# Don't change it
def find_best_split(feature, target):
    sorted_ind = np.argsort(feature)
    feature_sorted = feature[sorted_ind]
    target_sorted = target[sorted_ind]
```

```
    best_mse = float('inf')
    best_split = None

    for i in range(1, len(feature_sorted)):
        if feature_sorted[i] == feature_sorted[i - 1]:
            continue
        split_val = (feature_sorted[i] + feature_sorted[i - 1]) / 2
        left_target = target_sorted[:i]
        right_target = target_sorted[i:]
        left_mse = np.mean((left_target - np.mean(left_target))**2)
        right_mse = np.mean((right_target - np.mean(right_target))**2)
        # Compute weighted average of MSEs
        total_mse = (left_mse * len(left_target) + right_mse * len(right_
        if total_mse < best_mse:
            best_mse = total_mse
            best_split = split_val
    return best_split, best_mse
```

In [82]:
```
### your code starts here
best_hiv_split, best_hiv_mse = np.round(find_best_split(hiv, expectancy),
print("Best Split for HIV/AIDS:", best_hiv_split, ", best mse:", best_hiv
best_income_split, best_income_mse = np.round(find_best_split(income, exp
print("Best Split for Income:", best_income_split, ", best mse:", best_in
```

```
Best Split for HIV/AIDS: 0.65 , best mse: 38.194
Best Split for Income: 0.632 , best mse: 37.044
```

Which one do you think is the best feature for the first split? Why?

Income is the optimal feature for the first split because the MSE is lower.

In [83]:
```
# Run this to see if your answer is correct!
if best_hiv_mse < best_income_mse:
    chosen_name = 'HIV/AIDS'
    another_name = 'Income Composition of Resources'
    chosen_split = best_hiv_split
    chosen_feature = hiv
    another_feature = income
else:
    chosen_name = 'Income Composition of Resources'
    another_name = 'HIV/AIDS'
    chosen_split = best_income_split
    chosen_feature = income
    another_feature = hiv

print(f"Best first split is on {chosen_name} at {chosen_split:.2f}")
```

```
Best first split is on Income Composition of Resources at 0.63
```

We've successfully made the first split for our regression tree! The data is now divided into two parts: the left subtree and the right subtree. To grow the tree further, we need to determine which feature to use for splitting the left and right subtrees.

In [89]:
```
# Just run it. Don't change!
mask = chosen_feature <= chosen_split
left_feature_chosen, left_feature, left_target = chosen_feature[mask], an
right_feature_chosen, right_feature, right_target = chosen_feature[~mask]

left_split, left_mse = find_best_split(left_feature, left_target)
```

```
left_split_chosen, left_mse_chosen = find_best_split(left_feature_chosen,
right_split, right_mse = find_best_split(right_feature, right_target)
right_split_chosen, right_mse_chosen = find_best_split(right_feature_chos

print(f"Left Split for {another_name}: {left_split:.2f}, Best MSE: {left_
print(f"Left Split for {chosen_name}: {left_split_chosen:.2f}, Best MSE:
print(f"Right Split for {another_name}: {right_split:.2f}, Best MSE: {rig
print(f"Right Split for {chosen_name}: {right_split_chosen:.2f}, Best MSE
```

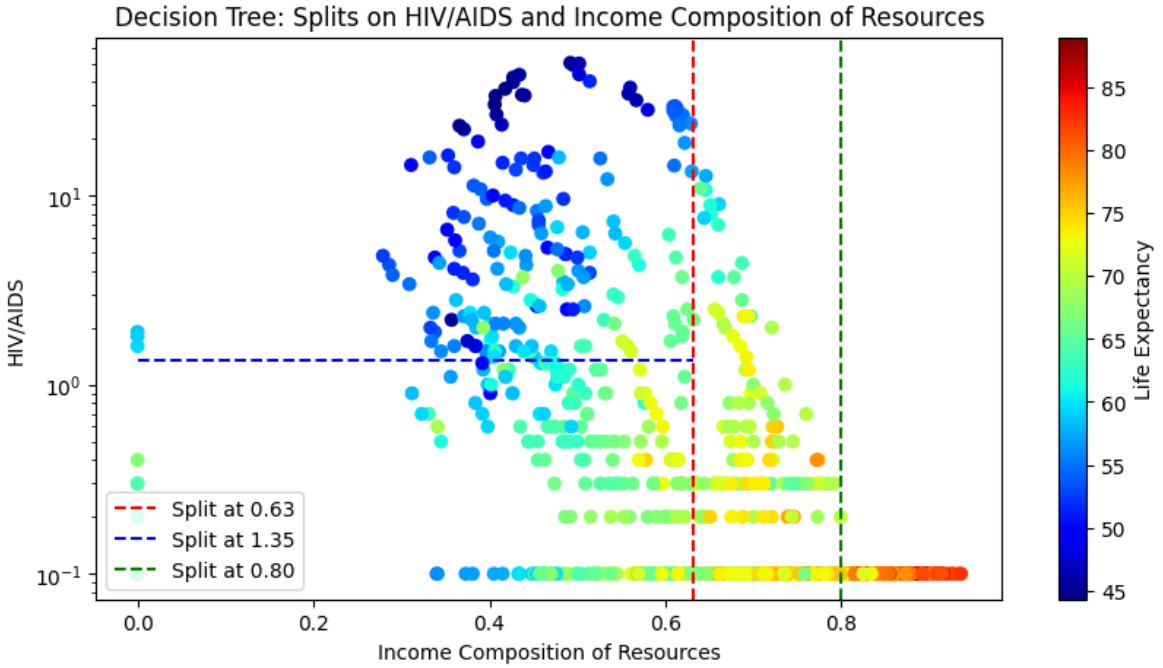
```
Left Split for HIV/AIDS: 1.35, Best MSE: 27.51
Left Split for Income Composition of Resources: 0.51, Best MSE: 40.13
Right Split for HIV/AIDS: 0.25, Best MSE: 20.14
Right Split for Income Composition of Resources: 0.80, Best MSE: 13.25
```

Based on the above results, find the correct feature (`left_name` and `right_name`) and split points for left subtree and right subtree (`left_split` and `right_split`).

In [91]:
```
### your code starts here, not necessarily one line for each
left_name = "HIV/AIDS"
left_split = 1.35
right_name = "Income Composition of Resources"
right_split = 0.8
print(f"Left Split on {left_name} at {left_split:.2f}. Right Split on {ri
```

```
Left Split on HIV/AIDS at 1.35. Right Split on Income Composition of Resou
rces at 0.80
```

In [95]:
```
plt.figure(figsize=(10, 5))
plt.scatter(chosen_feature, another_feature, c=expectancy, cmap='jet')
plt.axvline(x=chosen_split, color='red', linestyle='--', label=f'Split at
if left_name == chosen_name:
    plt.axvline(x=left_split, color='blue', linestyle='--', label=f'Split
else:
    plt.hlines(y=left_split, xmin=min(chosen_feature), xmax=chosen_split,
if right_name == chosen_name:
    plt.axvline(x=right_split, color='green', linestyle='--', label=f'Spl
else:
    plt.hlines(y=right_split, xmin=chosen_split, xmax=max(chosen_feature)
plt.colorbar(label='Life Expectancy')
plt.xlabel(chosen_name)
plt.ylabel(another_name)
if chosen_name == 'HIV/AIDS':
    plt.xscale("log")
else:
    plt.yscale("log")
plt.legend(loc='lower left')
plt.title('Decision Tree: Splits on HIV/AIDS and Income Composition of Re
plt.show()
```

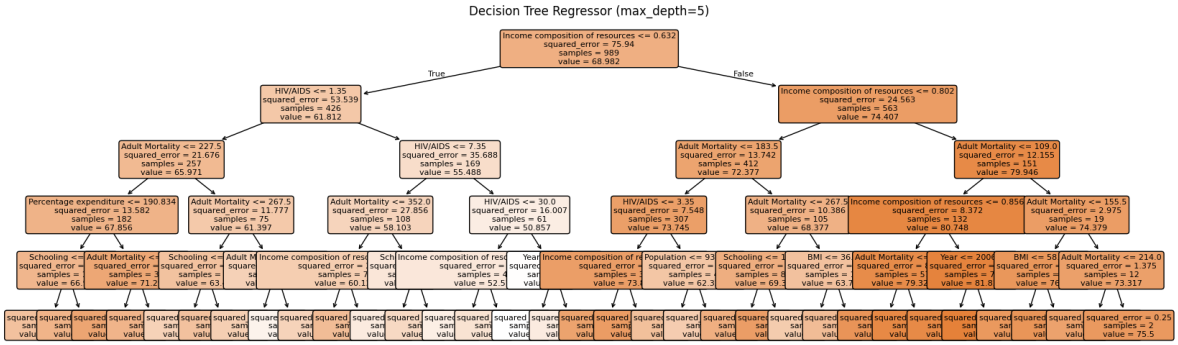


In [ ]:

1.2 Fitting a full regression tree and plot it

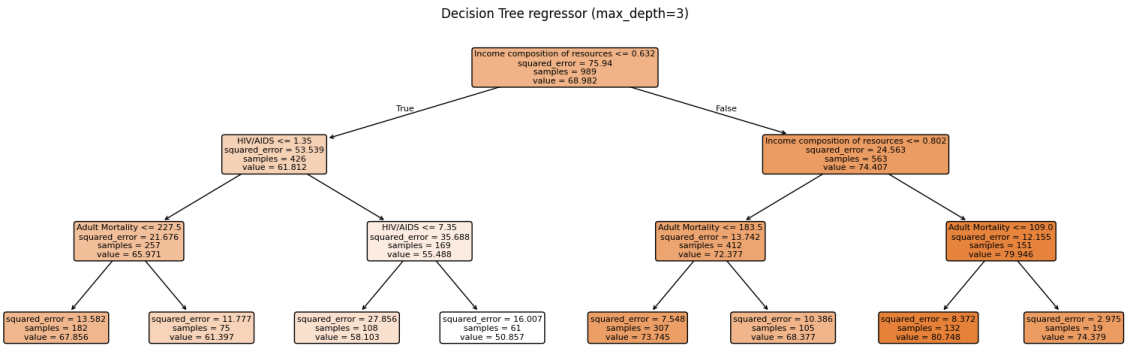
Now build a tree that uses all the predictor variables, has a more flexible structure, and is validated with a test set. Remember that we have split the full dataset into a training set and a test set. Fit a regression tree to the training set using the function `DecisionTreeRegressor` from `sklearn.tree`. For now, use your best judgment to choose parameters for tree complexity. Plot your decision tree use `plot_tree` function.

In [96]:
```
from sklearn import tree
### Your code starts here
# tree parameters go inside the first set of parentheses and the training
# goes in the second set of parentheses
dtree_full = tree.DecisionTreeRegressor(max_depth=5)
dtree_full.fit(X_train, y_train)
plt.figure(figsize=(20, 6))
tree.plot_tree(dtree_full,
feature_names=X_train.columns.tolist(),
filled=True,
rounded=True,
fontsize=8)
plt.title("Decision Tree Regressor (max_depth=5)")
plt.show()
```



What can you observe from this tree? If you set the `max_depth = 3`, is it consistant with what you find in problem 1.1b?

In [97]:
```
dtree = tree.DecisionTreeRegressor(max_depth=3)

### Your code starts here. Fit the tree and plot again!

dtree = tree.DecisionTreeRegressor(max_depth=3)
dtree.fit(X_train, y_train)
plt.figure(figsize=(20, 6))
tree.plot_tree(dtree,
feature_names=X_train.columns.tolist(),
filled=True,
rounded=True,
fontsize=8)
plt.title("Decision Tree regressor (max_depth=3)")
plt.show()
```



HIV and income are at the top as they are the strongest predictors. A max depth of 3 is consistent with the first tree.

1.3 Evaluation of your tree

Evaluate your regression tree with the test dataset. What is the R-squared and the MSE of the model using the test set? The `.predict` method for your model can help with this.

In [ ]:
```
# Making predictions on the test dataset
### Your code starts here
```

1.4 Let's try random forests

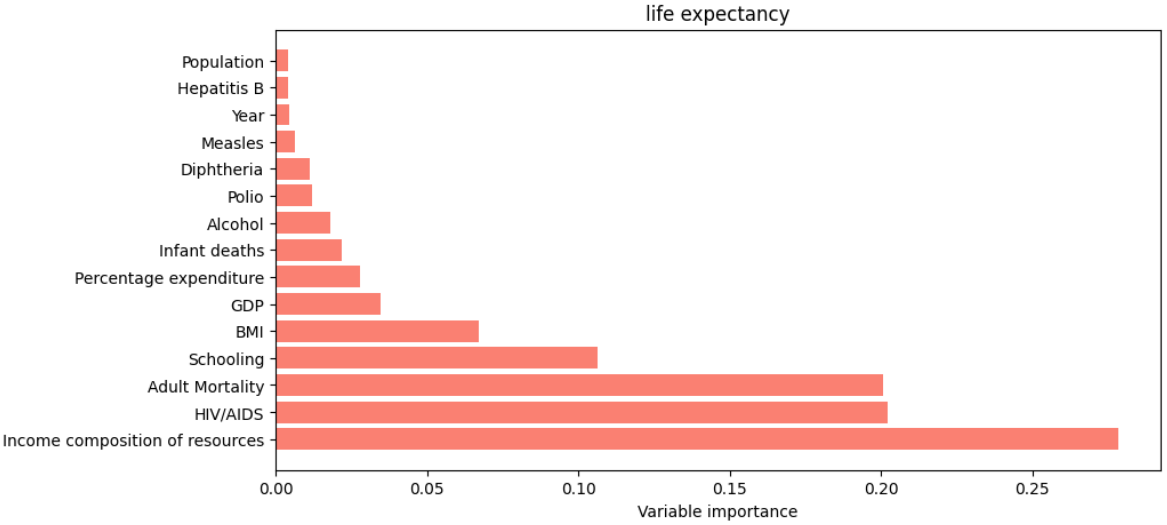
Finally, we will grow random forests to analyze the data, using the `RandomForestRegressor` function from `sklearn.ensemble`. Again, please use your best judgment to choose the initial parameters for tree complexity. Finish the following code to look at the "out of bag" (OOB) error as we increase the number of trees in the ensemble.

In [99]:
```
from sklearn import ensemble
from tqdm import tqdm
oob_error = []
num_trees = np.arange(50, 1000, 50)
### Your code starts here
# Initialize Random Forest Regressor with OOB Score enabled
rf = ensemble.RandomForestRegressor(
    max_features=4,
    min_samples_leaf=5,
    criterion='friedman_mse',
    oob_score=True,
)
# Loop over the range of number of trees

for m in tqdm(num_trees):
    rf.set_params(n_estimators=m)
    model = rf.fit(X, y)
    oob_error.append(1-model.oob_score_)
# Plotting the OOB error as the number of trees varies
plt.plot(num_trees, oob_error, marker='o')
plt.title('OOB error')
_ = plt.xlabel('Number of trees')
```

```
100%|██████████| 19/19 [01:05<00:00,  3.45s/it]
```

Visualize the variable importance of the model, which is the improvement in the loss (Gini index) due to splitting on a given variable, averaged over all of the trees in the forest. (The `.feature_importances_` method of the model may help with this.)

In [100…
```
rf.set_params(n_estimators=500)
model = rf.fit(X_test, y_test)

### Your code starts here

p = X.shape[1]
fig, ax = plt.subplots(figsize=(10,5))
inds = np.argsort(model.feature_importances_)
inds = np.flip(inds)
ax.barh(np.arange(p), model.feature_importances_[inds], color='salmon')
ax.set_yticks(np.arange(p))
_ = ax.set_yticklabels(X.columns[inds])
_ = ax.set_xlabel('Variable importance')
_ = ax.set_title('life expectancy')
```



Answer the following questions to evaluate the performance of random forests:

1. Which variables are most important in predicting life expectancy?
2. What test MSE do you obtain, and how does it compare to the test MSE of the regression tree above?

In [102…
```
from sklearn.metrics import mean_squared_error
y_pred_rf = model.predict(X_test)
print(f"Random Forest Test MSE: {mean_squared_error(y_test, y_pred_rf):.4
print(f"Decision Tree Test MSE: {mean_squared_error(y_test, dtree_full.pr
```

```
Random Forest Test MSE: 3.3362
Decision Tree Test MSE: 8.1058
```

income composition, hiv/aids, and adult mortality are the most important in predicting life expectancy. the forest test mse is lower than the regression tree test mse.

Problem 2: Penguin Culmen Analysis (PCA) (25 points)

Let's revisit our flightless friends with the new tools we've learned.

In [103…
```
# run the cell to import needed packages

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

%matplotlib inline
```

In [104…
```
# just run this cell to read in the data

df = pd.read_csv('https://raw.githubusercontent.com/YData123/sds265-sp26/
df = df.drop(columns=['index','year','island'])
df = df.dropna(axis=0)

# encode the labels
species = list(set(df['species']))
df['class'] = LabelEncoder().fit_transform(df['species'])
sex = [int(list(df['sex'])[j]=='male') for j in range(len(df))]
df['sex'] = sex
df = df.drop(columns=['species'])

y = np.array(df['class'])
X = df.copy()
X = X.drop(columns=['class'])
```

In [105…
```
# just run this cell to standardize

for i in range(5):
    X.iloc[:,i] = (X.iloc[:,i]-np.mean(X.iloc[:,i]))/np.std(X.iloc[:,i])

print(f"X is {X.shape[0]} rows with {X.shape[1]} columns.")
```

```
X is 333 rows with 5 columns.
```

```
/tmp/ipykernel_919/619360440.py:4: FutureWarning: Setting an item of incom
patible dtype is deprecated and will raise in a future error of pandas. Va
lue '0      0.991031
1     -1.009050
2     -1.009050
4     -1.009050
5      0.991031
         ...
339    0.991031
340   -1.009050
341    0.991031
342    0.991031
343   -1.009050
Name: sex, Length: 333, dtype: float64' has dtype incompatible with int64,
please explicitly cast to a compatible dtype first.
  X.iloc[:,i] = (X.iloc[:,i]-np.mean(X.iloc[:,i]))/np.std(X.iloc[:,i])
```

2.1 Run PCA

In the next cell, carry out Principle Component Analysis to reduce the data from 5 dimensions to 2.

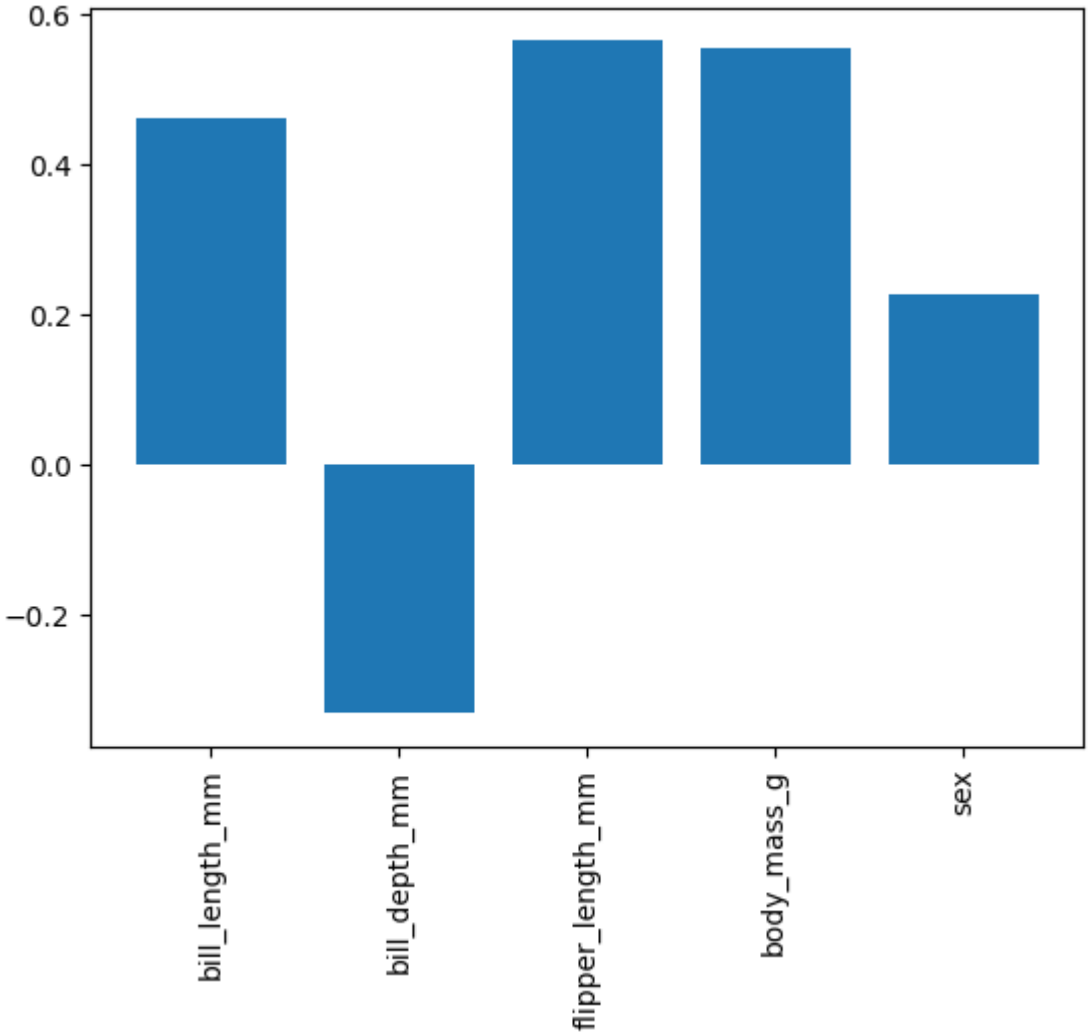
Let `pv1` be the first principal vector and let `pv2` be the second principal vector. Let `pcs` be the projection of the data onto the first two principal vectors.

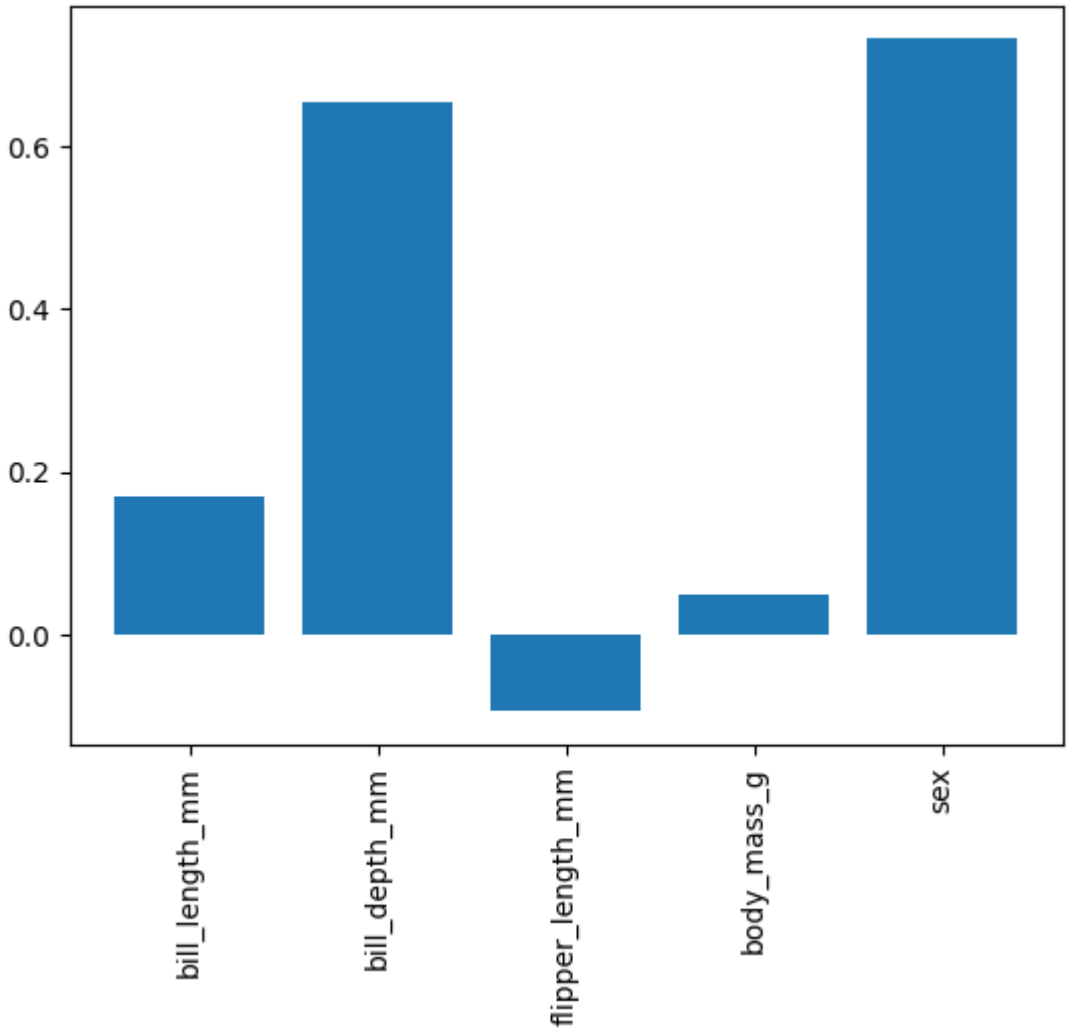
In [106…
```
# your code here
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pcs = pca.fit_transform(X)

pv1 = pca.components_[0]
pv2 = pca.components_[1]
```

2.2 Principle Component Analysis

The next cell plots the principal vectors.

In [107…
```
# just run this cell

plt.bar(range(5), pv1)
plt.xticks(range(5), X.columns, rotation='vertical')
plt.show()

plt.bar(range(5), pv2)
plt.xticks(range(5), X.columns, rotation='vertical')
plt.show()
```


0.6
0.4
0.2
0.0
−0.2
bill_length_mm
bill_depth_mm
flipper_length_mm
body_mass_g
sex



Are the first two principle vectors orthogonal? Write code to check and also explain conceptually.

Looking at the plot, what are the first two principle components capturing, in terms of the original features we have?

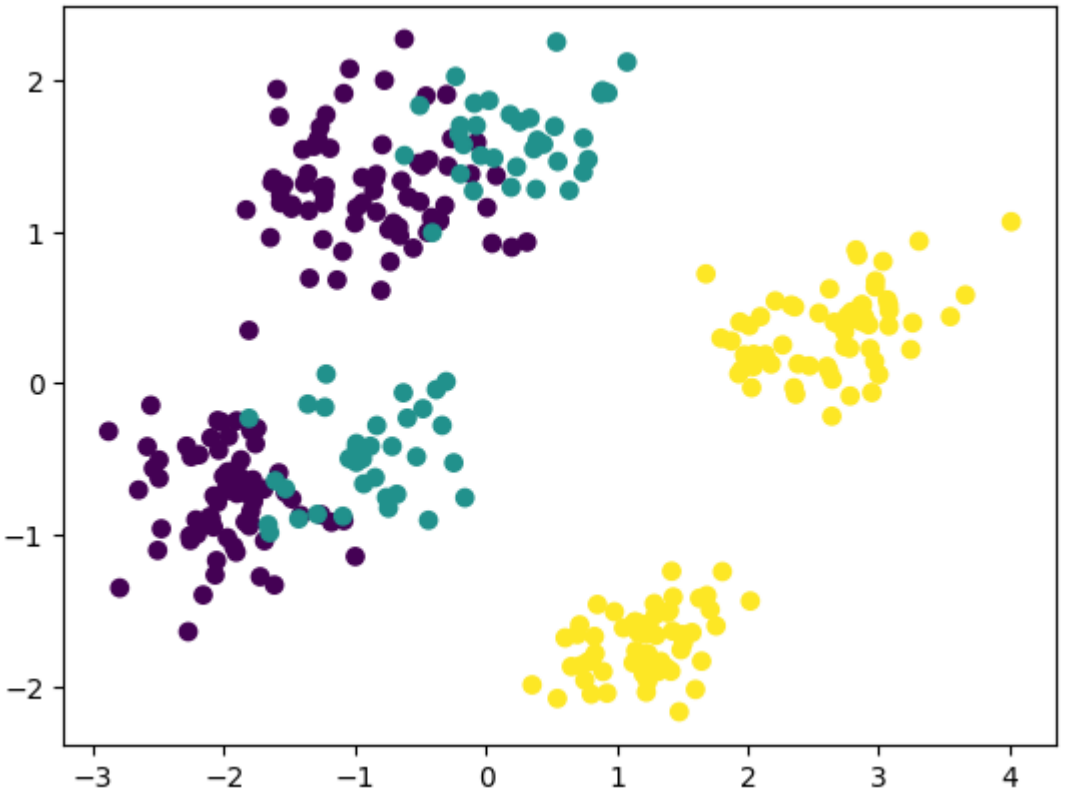
In [108…
```
# your code here
dot_product = np.dot(pv1, pv2)
print(f"Dot product of pv1 and pv2: {dot_product:.6f}")
```

```
Dot product of pv1 and pv2: -0.000000
```

[your markdown here]

2.3 Visualization

In [109…
```
plt.scatter(pcs[:, 0], pcs[:, 1], c = y)
plt.show()
```



What is being plotted in this cell above? If you are to add an xlabel and ylabel, what would you call them?

Can you explain why there are pretty much 6 clusters in the plot?

Recall when we plotted using two features of your choice in assignment 2. How are these different and similar?

Each penguin is plotted in 2D PCA space, colored by species. The axes would be "Principal Component 1" and "Principal Component 2."

pv1 and pv2 are orthoginal because np.dot(pv1, pv2) ≈ 0

There are six groups because species × 2 sexes = 6 groups. PC1 separates species (body size), PC2 separates males from females within each species.

Both plots show 2D structure in the data, but PCA uses all 5 features optimally while Assignment 2 used just two raw features. PCA tends to give cleaner separation since no information is discarded; the tradeoff is that the axes are harder to interpret directly.

2.4 Visualize the decision boundries

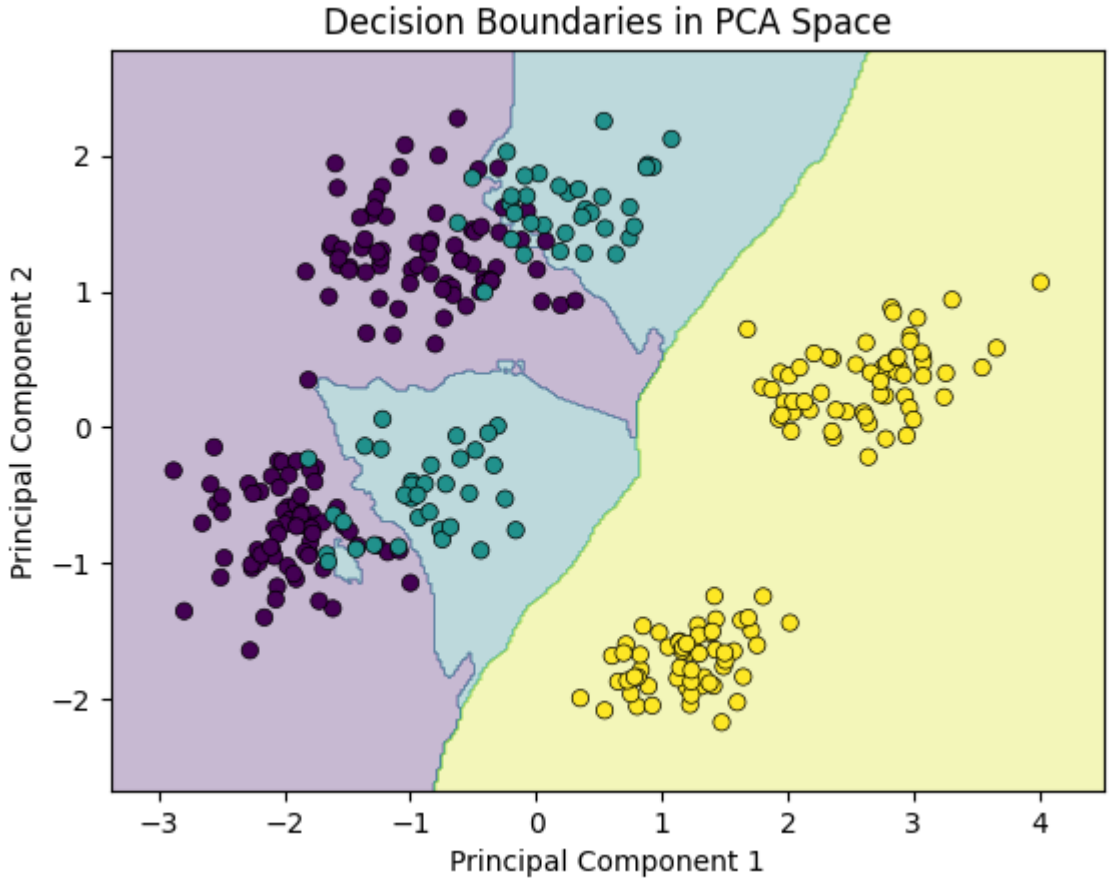
We now want to add the decision boundries to the plot. (Similar to what we did in assignment 2, but this time using the first 2 principle components as x and y.)

In [112…
```
# just run this cell
# the function will again help you plot the decision boundries
```

```
def plot_decision_boundaries(X, y, lr, error):
    X2 = np.array(X)
    b = lr.intercept_
    beta = lr.coef_.T
    colors = ['orange', 'pink', 'lightgreen']
    h = 0.015
    x_min, x_max = X2[:, 0].min() - .5, X2[:, 0].max() + .5
    y_min, y_max = X2[:, 1].min() - .5, X2[:, 1].max() + .5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))
    Z = np.dot(np.c_[xx.ravel(), yy.ravel()], beta) + b
    Z = np.argmax(Z, axis=1)
    Z = Z.reshape(xx.shape)
    fig = plt.figure(figsize=(8,8))
    plt.contourf(xx, yy, Z, levels=[0,.5,1.5,2.5], colors=colors, alpha=0
    for c in range(3):
        mask = (y==c)
        plt.scatter(X2[np.array(mask),0], X2[np.array(mask),1], color=col

    plt.xlim(xx.min(), xx.max())
    plt.ylim(yy.min(), yy.max())
    plt.legend(loc='upper left')
    plt.title('error rate: %.2f' % error)
    plt.xlabel(X.columns[0])
    plt.ylabel(X.columns[1])
```

In [111…

```
from sklearn.neighbors import KNeighborsClassifier

# Train classifier on PCA-reduced data
clf = KNeighborsClassifier()
clf.fit(pcs, y)

# Create mesh grid over the PCA space
x_min, x_max = pcs[:, 0].min() - 0.5, pcs[:, 0].max() + 0.5
y_min, y_max = pcs[:, 1].min() - 0.5, pcs[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))

# Predict over the grid and plot
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.3)
plt.scatter(pcs[:, 0], pcs[:, 1], c=y, edgecolors='k', linewidths=0.5)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Decision Boundaries in PCA Space")
plt.show()
```



How do you think the model performed comparing to the models you ran in assignment 2?

It likely performed similarly or better than the best model from Assignment 2, despite using only 2 dimensions instead of 5.

Problem 3: PCA Applications -- Faces and Genetics data (25 points)

In this problem, we will apply PCA to two applications. In the first, we will apply PCA to analyze some face image data and try to understand the variation in this data. In the second application, we will apply PCA to some genetics data and recover an interesting connection with geography!

In [113…
```
import numpy as np
from sklearn.decomposition import PCA
```

Problem 3.1: Face image data

In this problem, we will apply implementation of PCA to analyze face image data from the `olivetti_faces` dataset. The dataset consists of 64×64 greyscale images of faces taken in a controlled setting, varying lighting, facial expressions (eyes open/closed, smiling/not smiling), and facial details (glass / no glasses). The dataset consists of 40 subjects with 10 images for each. It was collected by AT&T between 1992 and 1994.

We will try to understand the variation in face (images) by performing principal component analysis!

First, we fetch the dataset and define a utility function for plotting a gallery of images.

In [114…
```
# this cell fetches the data and defines a function to plot the images

import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces

# get data
faces, _ = fetch_olivetti_faces(return_X_y=True)

n_samples, n_features = faces.shape
image_shape = (64, 64)

# define a utility function to plot a gallery of images
def plot_gallery(title, images, n_col=3, n_row=2, cmap=plt.cm.gray):
    fig, axs = plt.subplots(
        nrows=n_row,
        ncols=n_col,
        figsize=(2.0 * n_col, 2.3 * n_row),
        facecolor="white",
        constrained_layout=True,
    )
    fig.set_constrained_layout_pads(w_pad=0.01, h_pad=0.02, hspace=0, wsp
    fig.set_edgecolor("black")
    fig.suptitle(title, size=16)
    for ax, vec in zip(axs.flat, images):
        im = ax.imshow(
            vec.reshape(image_shape),
            cmap=cmap,
            interpolation="nearest",
        )
        ax.axis("off")

    fig.colorbar(im, ax=axs, orientation="horizontal", shrink=0.99, aspec
    plt.show()
```

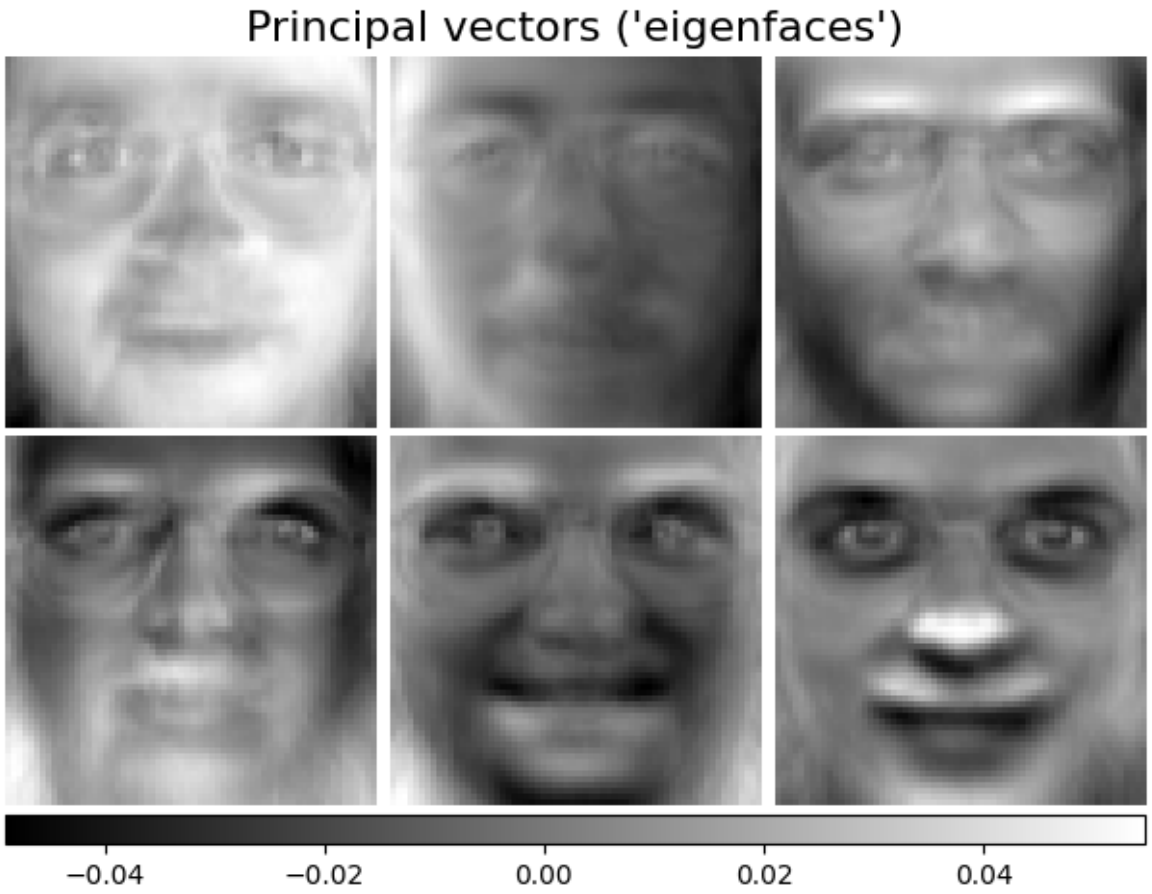
In [115…
```
# plot first 6 images in dataset
plot_gallery("First 6 images", faces[:6, :])
```

Problem 3.1.1: Fit PCA and visualize the top principal vectors

In [116…
```
# fit PCA to the `faces` data. Use `n_components=None` to compute the max
# Cell 2: fit PCA
from sklearn.decomposition import PCA

pca = PCA(n_components=None)
pca.fit(faces)
```

Out[116…
```
▾ PCA i ?

PCA()
```

In [117…
```
# extract the principal vectors from the fitted PCA object
# fit PCA to the `faces` data. Use `n_components=None` to compute the max
# Cell 3: extract principal vectors and plot eigenfaces
face_principal_vectors = pca.components_
plot_gallery("Principal vectors ('eigenfaces')", face_principal_vectors)
```



Problem 3.1.2: Visualize explained variance of the top k principal vectors

When we compute PCA, we can get the "explained variance" of each principal vector. In this problem, we will visualize the explained variance to better-understand the variation in the data.

Recall that in `sklearn` 's implementation of PCA, the explained variance ratio of the i th principal vector is given by `pca.explained_variance_ratio_[i]` . This is a number between 0 and 1. You can think of this as the percentage of the variation in the data in the direction of the i th principal vector.

Then, we can define the cumulative explained variance of the top k principal vectors as,

$$\text{cumulative\_explained\_variance\_ratio}(k) = \sum_{i=1}^{k} \text{explained\_variance\_ratio}(i).$$

This quantity represents the proportion of the variation in the data which is explained by the first k principal vectors. In this problem, we will plot k against against $\text{explained\_variance\_ratio}(k)$ and $\text{cumulative\_explained\_variance\_ratio}(k)$. This gives us an idea of how many directions account for most of the variation in the data.

Aside: How is the "explained variance ratio" obtained?

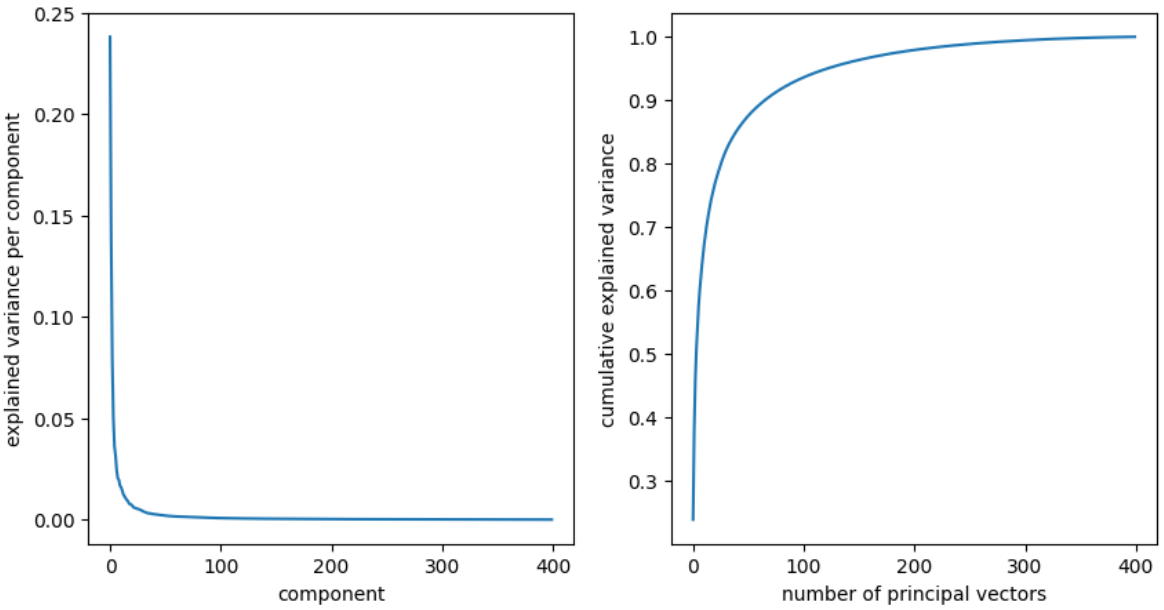
Recall that PCA is computed via the eigenvectors and eigenvalues of the empirical

covariance matrix. Let S be the empirical covariance matrix. Then, its eigenvectors v_i are the principal vectors. Denote the eigenvalue of the i th eigenvector by λ_i. In `sklearn`, the eigenvalues are accessible via `pca.explained_variance_`; λ_i is `pca.explained_variance_[i]`. The "explained variance ratio" is a ratio of each eigenvalue to the sum of the eigenvalues. That is, $\text{explained\_variance\_ratio}(i) = \lambda_i / \sum_j \lambda_j$.

The eigenvalue of a particular eigenvector of the covariance matrix will be larger when there is more variation in that direction. The "ratio" part of explained variance simply normalizes these eigenvalues so that they sum to 1, giving the quantity an interpretation as the proportion of the variance explained by a particular direction.

In [118…
```
# plot the explained variance of the top k principal components as a func

# extract the explained variance ratio of each principal vector
explained_variance_ratio_per_component = pca.explained_variance_ratio_

# compute cumulative explained variance ratio upto k components for each
cumulative_explained_variance_ratio = np.cumsum(explained_variance_ratio_

# plot the explained variance ratio per component and the cumulative expl
fig, (ax1, ax2) = plt.subplots(figsize=(10, 5), ncols=2)
ax1.plot(explained_variance_ratio_per_component)
ax1.set_xlabel('component')
ax1.set_ylabel('explained variance per component')

ax2.plot(cumulative_explained_variance_ratio)
ax2.set_xlabel('number of principal vectors')
ax2.set_ylabel('cumulative explained variance')
```

Out[118…
```
Text(0, 0.5, 'cumulative explained variance')
```



Comment on the curve you observe. Roughly how many principal vectors are required to explain 90% of the variance? What does this tell you about the underlying variation in the data?

[Your markdown here]

The per-component explained variance drops off very rapidly; the first few components capture a disproportionately large share of the variance, and contributions quickly become negligible. The cumulative curve rises steeply at first, then flattens into a plateau approaching 1.

k_90 = np.searchsorted(cumulative_explained_variance_ratio, 0.90) + 1
print(f"Components needed for 90% variance: {k_90}")

Problem 3.1.3: Reconstructing data from principal components

In this question, we will use PCA to re-construct the face images from fewer principal components. The plots above show that most of the variance in the data is explained by the first few principal components. In this question, we will visualize this by reconstructing a face image using the first k principal components, varying k. We will then use the face image PCA to (attempt to) reconstruct an image of an octopus.

Recall that the first k principal components, $v_1, \ldots, v_k$ give an orthonormal basis for a k-dimensional subspace of the data. The first k principal components of a point $x \in \mathbb{R}^d$ are given by $\left(v_1^\top (x - \bar{x}), \ldots, v_k^\top (x - \bar{x})\right)$. Each principal component represents the amount of x that lies in the direction of the corresponding principal vector. The principal components can then be used to reconstruct x via,

$$\hat{x} = \bar{x} + \sum_{i=1}^{k} \left(v_i^\top (x - \bar{x})\right) v_i.$$

This is essentially a projection onto the k-dimensional subspace spanned by the principal vectors $v_1, \ldots, v_k$ (after centering by the mean).

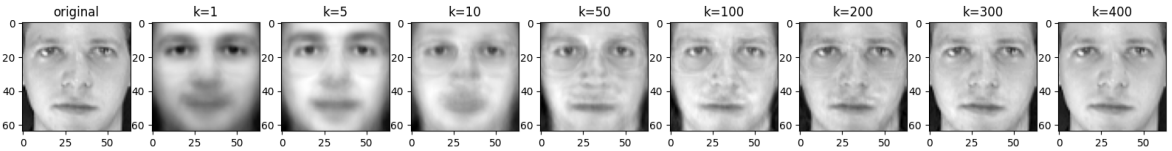
In the `sklearn` implementation of PCA, the mean vector $\bar{x}$ is accessible via `pca.mean_` and the principal vectors are accessible by `pca.compnents_` .

Implement a function `project_principal_vectors` which computes $\hat{x}$ for a given x, reconstructing a point in terms of the principal vectors.

In [119…
```
def project_principal_vectors(x, principal_vecs, mean_x, k):
    """
    project a data point onto the first k principal vectors

    Parameters
    ----------
    x : np.ndarray of shape (d,)
        test data point
    principal_vecs : np.ndarray of shape (min(d, n), d)
        array of eigenvectors where eig_vecs[i] is the ith eigenvector
    mean_x : np.ndarray of shape (d,)
        the mean vector of the data
    k : int
        number of principal vectors to project onto

    Returns
```

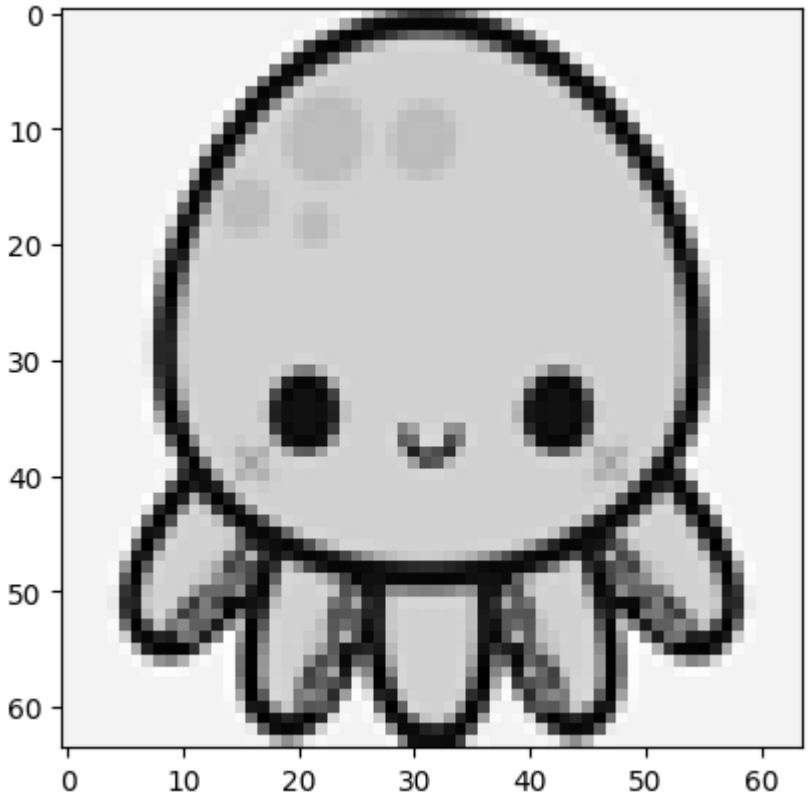
```
    -------
    np.ndarray of shape (d,)
        the projection of x onto the first k principal vectors
    """

    # center the data point
    centered_x = x - mean_x # your code here

    # compute the projection onto the first k principal components
    projection = mean_x + sum([
        (principal_vecs[i] @ centered_x) * principal_vecs[i] # your code
        for i in range(k)])

    return projection
```

In [120…
```
# extract mean vector and principal components from fitted PCA object
face_mean_x = pca.mean_
face_principal_vectors = pca.components_

face = faces[0]

fig, axs = plt.subplots(figsize=(20,6), ncols=9)

# plot the original image
axs[0].imshow(face.reshape(image_shape), cmap='gray')
axs[0].set_title('original')

for ax, k in zip(axs[1:], [1, 5, 10, 50, 100, 200, 300, 400]):
    reconstruction = project_principal_vectors(face, face_principal_vecto
    ax.imshow(reconstruction.reshape(image_shape), cmap='gray')
    ax.set_title(f'k={k}')
```

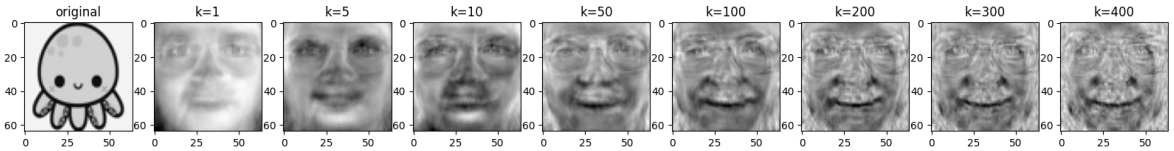


What happens if we try to apply our PCA algorithm, which was fitted on images of faces, to an image of something that is not a face? Let's try! Below, we'll attempt to use PCA to reconstruct an image of a cute octopus.

In [121…
```
# source of image: "https://i.pinimg.com/originals/6d/dc/5a/6ddc5a4845441
# the cute_octopus.png file is a cropped, reshaped, and grayscale version
from PIL import Image
import urllib

url = 'https://raw.githubusercontent.com/YData123/sds265-sp26/main/assign
cute_octopus = np.array(Image.open(urllib.request.urlopen(url)))
plt.imshow(cute_octopus, cmap='gray')
cute_octopus = cute_octopus.flatten() # flatten the image into a vector
```



In [122…
```
# extract mean vector and principal components from PCA object fitted to
face_mean_x = pca.mean_
face_principal_vectors = pca.components_

fig, axs = plt.subplots(figsize=(20,6), ncols=9)

# plot the original image
axs[0].imshow(cute_octopus.reshape(image_shape), cmap='gray')
axs[0].set_title('original')

for ax, k in zip(axs[1:], [1, 5, 10, 50, 100, 200, 300, 400]):
    reconstruction = project_principal_vectors(cute_octopus, face_princip
    ax.imshow(reconstruction.reshape(image_shape), cmap='gray')
    ax.set_title(f'k={k}')
```



What do you observe about the reconstruction of the face image? How well do the reconstructed images match the original image at each k. What about the reconstruction of the octopus? Why do you think this is what we observe?

[Your markdown here]

**** Quality improves progressively with k — blurry and unrecognizable at k=1–5, broadly face-like by k=10–50, and nearly perfect around k=100–200. This is consistent with the explained variance curve: ~90% of variance is captured by ~100 components, so most meaningful structure is recoverable well before using all 400.**

Even at k=400, the reconstruction never resembles the original; instead it looks like

a ghostly, face-like distortion. This is because the face PCA basis only spans directions of variation present in face images; the octopus contains structure (tentacles, texture, body shape) that lies largely outside this subspace. The reconstruction is simply the closest point to the octopus *within the face subspace*, which looks like a distorted face. This illustrates that PCA is data-dependent; its basis is only meaningful for data similar to what it was fitted on.

Problem 3.2: Analyzing genetics data

In this sub-problem, we will PCA to analyzing genetics data. The data we will use comes from the 1000 Genomes Project.

A single-nucleotide polymorphism is a substitution of a single nucleotide at a specific location in the genome which is present in a sufficiently large segment of the population. An ancestry-informative SNP (AISNP) is a SNP which has significant variation across global populations. There exists a line of work identifying AISNPs. In this problem, we will use 55 AISNPs identified by Kidd et al.. We have pre-processed this data for you.

In this problem, you will use PCA to visualize and analyze this data.

In [123…
```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import OneHotEncoder
from sklearn.decomposition import PCA
```

In [124…
```
# read in the SNP data
snp_data = pd.read_csv('https://raw.githubusercontent.com/YData123/sds265
snp_cols = [col for col in snp_data.columns if col.startswith('rs')]
sample_attr_cols = ['pop', 'super_pop', 'gender']
snp_data.head()
```

Out[124…

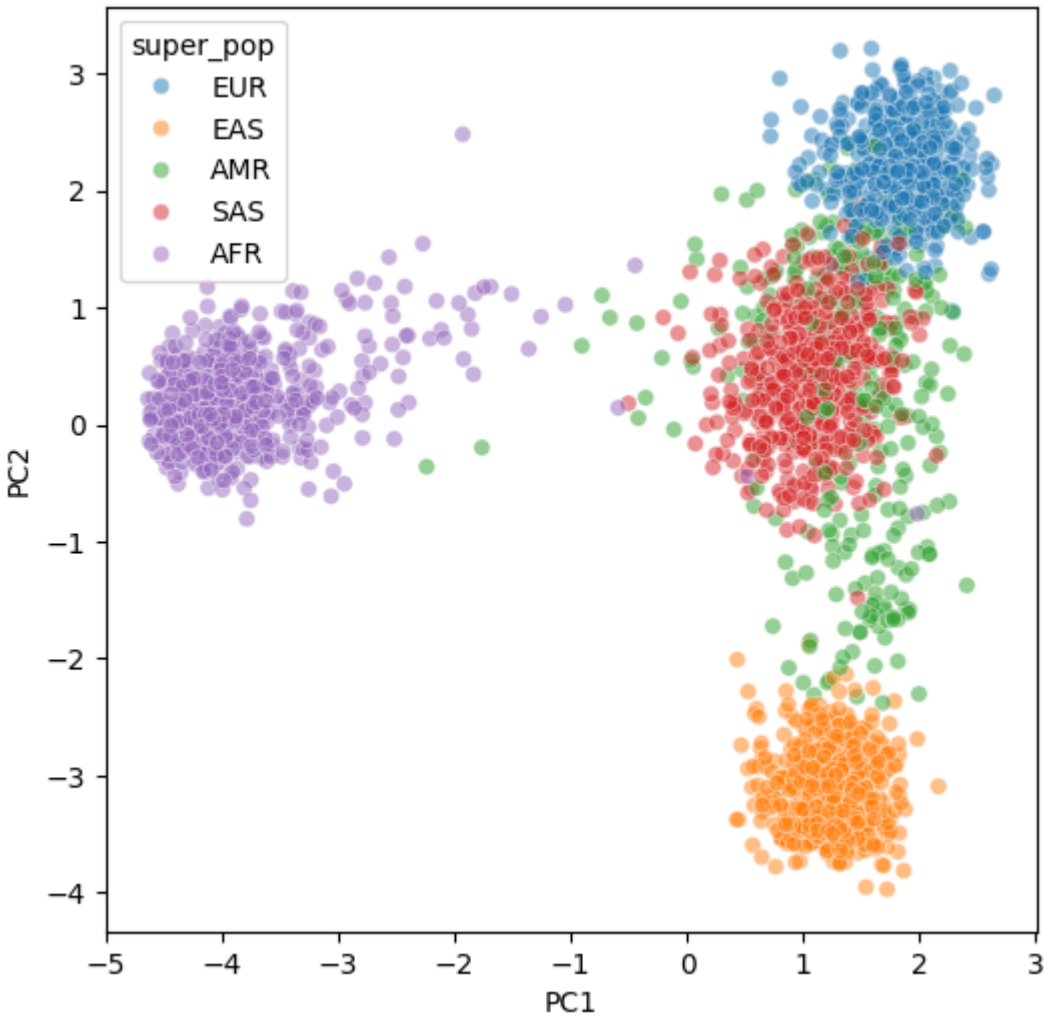
	rs3737576	rs7554936	rs2814778	rs798443	rs1876482	rs1834619	rs3827760	rs26
0	0	1	0	2	0	0	0	
1	0	2	0	2	0	0	0	
2	0	1	0	1	2	1	0	
3	0	2	0	2	0	0	0	
4	0	1	0	2	1	1	0	

5 rows × 58 columns

In [125…
```
# one-hot encode the SNP data using sklearn's OneHotEncoder for use with
import sklearn
try:
    ohe = OneHotEncoder(sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(sparse=False)
X = ohe.fit_transform(snp_data[snp_cols].values)
```

In [126…
```
# fit PCA on the snp data in X; set the number of components to 2
pca_snp = PCA(n_components=2)
pca_snp.fit(X)

# compute first two principal components of each sample in X
X_pca = pca_snp.transform(X)

# add the principal components to the snp_data dataframe
snp_data['PC1'] = X_pca[:, 0]
snp_data['PC2'] = X_pca[:, 1]
```

In [127…
```
# plot the principal components in a scatter plot with super population c

fig, ax = plt.subplots(figsize=(6, 6))
sns.scatterplot(data=snp_data, x='PC1', y='PC2', hue='super_pop', alpha=0
```



Take a look at this plot and think about any trends you see. Do you see any connection with geography?

Below, we will rotate this plot so that the center of the EUR and AFR samples on the PC plot lie vertically on top of each other.

In [128…
```
# compute mean PC1 and PC2 for each super population
mean_by_superpop = snp_data.groupby('super_pop')[['PC1', 'PC2']].mean()
afr_pc_mean = mean_by_superpop.loc['AFR'].to_numpy()
eur_pc_mean = mean_by_superpop.loc['EUR'].to_numpy()
```

```
# compute angle between center of AFR and EUR clusters
angle = np.arctan2(eur_pc_mean[1] - afr_pc_mean[1], eur_pc_mean[0] - afr_
print(f'angle between AFR and EUR principal components: {angle:.2f} radia
```

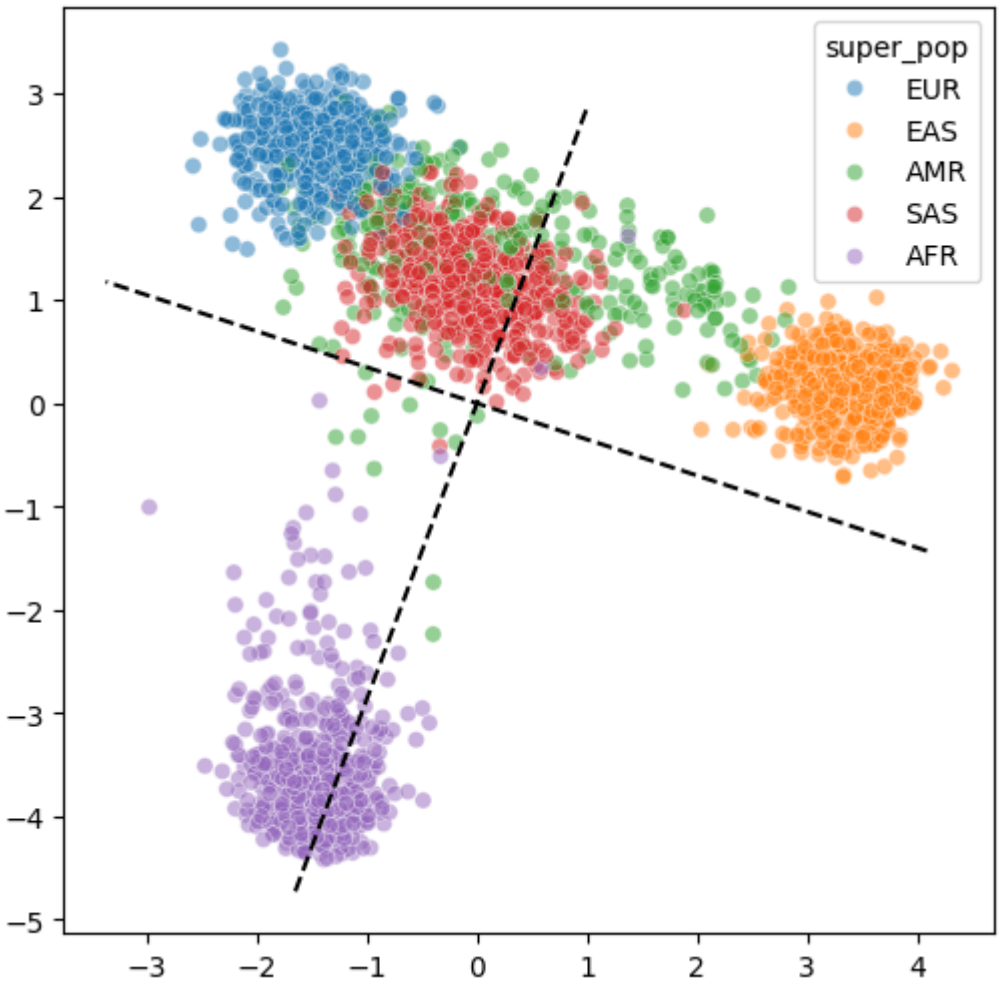
```
angle between AFR and EUR principal components: 0.34 radians
```

In [129…

```
def create_rotation_mat(angle):
    '''creates 2D rotation matrix for given angle'''
    c, s = np.cos(angle), np.sin(angle)
    rotation_matrix = np.array(((c, -s), (s, c)))
    return rotation_matrix

# rotate the principal components by the angle between the AFR and EUR cl
rotation_mat = create_rotation_mat(angle - np.pi/2)
rotated_pcs = snp_data[['PC1', 'PC2']].to_numpy() @ rotation_mat

# create lines denoting the principal components in the rotated axes
pc1_line = [(pc1, 0) for pc1 in ax.get_xlim()]
pc2_line = [(0, pc2) for pc2 in ax.get_ylim()]
pc1_line_rotated = pc1_line @ rotation_mat
pc2_line_rotated = pc2_line @ rotation_mat
```

In [130…

```
# plot rotated principal components
fig, ax = plt.subplots(figsize=(6, 6))
sns.scatterplot(x=rotated_pcs[:, 0], y=rotated_pcs[:, 1], hue=snp_data['s
ax.plot(pc1_line_rotated[:,0], pc1_line_rotated[:,1], color='black', line
ax.plot(pc2_line_rotated[:,0], pc2_line_rotated[:,1], color='black', line
```



For reference, below is a cartogram of the global population from wikipedia. It shows a

map of the world with land area scaled according to population.

Comment on any patterns you observe. Are "super populations" which are geographically closer also closer in terms of their first two principal components? Can you suggest any explanations for the patterns you observe?

[Your markdown here]

Solution: The five super-populations form well-separated clusters. SAS falls between EUR and EAS, and AMR sits centrally between multiple groups. AFR is most distant from all others, and also shows the most internal spread.

Additionally, geographically closer populations tend to be closer in PC space (e.g. EUR-SAS-EAS form a rough gradient along PC1), with a few exceptions explained by admixture.

Possible Explanations: The clusters in PC space closely mirror global geography: AFR is most separated from all others, EUR and EAS sit at opposite ends of PC1, SAS falls between them, and AMR sits centrally near multiple groups.*

This pattern reflects human migration history. Modern humans originated in Africa, so African populations have had the longest time to diversify — explaining AFR's large spread and separation from the rest. All non-African populations descend from a small group that migrated out of Africa, making them more genetically similar to each other than to AFR. From there, populations spread eastward, so EUR and EAS ended up most different from each other (opposite ends of PC1), with SAS naturally falling in between geographically and genetically. Finally, AMR sits centrally because Native Americans descended from East Asian ancestors via the Beringian migration, and were later admixed with European and African ancestry during colonization.

Problem 4: Bayes' Theorem (10 points)

Bayes' theorem provides a systematic way to update beliefs based on new evidence. It is a powerful tool in probability and statistics, and it is also useful for understanding many real-world phenomena.

Problem 4.1: Counterintuitive Example in Disease Prediction

Machine learning models are increasingly used in clinical situations. A common and counterintuitive point, often missed even by experienced doctors, is that for rare conditions, highly accurate tests can still produce many false positive results. Since false positives can trigger additional tests, costs, and anxiety, this fact has important implications for clinical practice. Screening and early detection policies often aim to balance benefits and harms, and may focus on high-risk groups. For example, X-ray examinations for breast cancer are often only recommended for women who are older than 40. Such an X-ray examination in young women can be more harmful.

The following calculation could help you understand the phenomenon:

For a woman in her 40s, the probability of having a breast cancer is 7 in 1,000 $P(P \mid D) = 0.7\%$. The X-ray test has a sensitivity of 0.9 and a false positive rate of 0.07, which means that 9 out of 10 women who truly have breast cancer will test positive, while 7 out of 100 women who do not have breast cancer will still test positive.

If a woman in her 40s tests positive, what is the probability that the woman really has the breast cancer?

[Your answer should be here]

P(D) = 0.007 P(¬D) = 0.993 P(pos|D) = 0.9 P(pos|¬D) = 0.07 P(pos) = P(pos | D)P(D) + P(pos | ¬D)P(¬D) P(pos) = (0.9)(0.007) + (0.07)(0.993) = 0.07581

P(D | pos) = P(pos | D)P(D)/P(pos) P(D | pos) = (0.9)(0.007)/(0.07581) ≈ 0.0831

~8.31%

Suggested further reading: Steven Strogatz, *"Chances Are"* (https://archive.nytimes.com/opinionator.blogs.nytimes.com/2010/04/25/chances-are/).

Problem 4.2 Dirichlet Distribution in Cancer Subtype

Accurately identifying cancer subtypes is crucial because subtype information can directly affect therapy choices and prognosis. However, subtype classification can be difficult in practice: different diagnostic methods have different advantages, and the most accurate methods often require a biopsy, which can harm patients. By contrast, noninvasive methods are usually less accurate. Therefore, it is important to choose the best method for the specific clinical situation.

Consider four common breast cancer subtypes: Luminal A, Luminal B, HER2-enriched, and Triple-negative. We assume no prior preference over the 4 subtypes:

Let $\boldsymbol{\theta} = (\theta_1, \theta_2, \theta_3, \theta_4)$ denote the cancer subtype probabilities.

We use a uniform Dirichlet prior:

$$\boldsymbol{\theta} \sim \mathrm{Dir}(1, 1, 1, 1).$$

In a specific region, we observe **20** independent cases with the following counts: Luminal A (10 cases), Luminal B (4 cases), HER2-enriched (4 cases), and Triple-negative (2 cases).

Then, **1)** compute the posterior distribution of $\boldsymbol{\theta}$ and **2)** There is a diagnostic test: for patients with the triple-negative subtype, the probability of testing positive is 0.9; for patients with other subtypes, the probability of testing positive is 0.1. The 21st patient tests positive, compute the posterior probability that the 21st patient belongs to the triple-negative subtype.

[Your answer should be here]

1.

$$\theta \mid \text{data} \sim \mathrm{Dir}(1 + 10,\ 1 + 4,\ 1 + 4,\ 1 + 2)$$

$$\theta \mid \text{data} \sim \mathrm{Dir}(11,\ 5,\ 5,\ 3)$$

2

$$P(pos \mid TN) = 0.9$$

$$P(pos \mid \text{other}) = 0.1$$

LOTP

$$P(pos) = 0.1\left(\frac{11}{24}\right) + 0.1\left(\frac{5}{24}\right) + 0.1\left(\frac{5}{24}\right) + 0.9\left(\frac{3}{24}\right) = 0.2$$

BAYE

$$P(TN \mid pos) = \frac{P(pos \mid TN)\ P(TN)}{P(pos)}$$

$$P(TN \mid pos) = \frac{(0.9)(3/24)}{0.2} = 0.5625$$